

DroidConsole - One-Page App Summary

Evidence used: README.md, DOCS.md, package.json files, docker-compose.yml, electron/app.js, desktop-host/src, web-frontend/src.

What It Is

DroidConsole is a Windows Electron desktop app for managing, mirroring, and controlling multiple Android devices from a PC. It combines a React UI, Express APIs, WebSocket video/control channels, ADB, scrcpy, and UHID input.

Who It's For

Primary user/persona: Windows PC operators or developers managing many Android devices over USB or ADB TCP. **Formal persona name:** Not found in repo.

What It Does

- **Multi-device mirroring:** auto-starts scrcpy sessions for connected Android devices.
- **Device control:** sends touch, keyboard, scroll, clipboard, DPAD, rotation, and power inputs.
- **Device organization:** nicknames, groups, group filtering, context menu moves, and master/slave broadcast.
- **Batch operations:** install APKs, push files, reboot, clear app data, screenshots, and shell commands.
- **WhatsApp workflows:** WA registration typing and warm-up flows with device selection and daily phase limits.
- **Remote sharing:** share codes, received-share view, video streaming, and WebRTC signaling endpoint.
- **TCP/WiFi connect:** connects Android devices wirelessly through ADB TCP.

How It Works

Component	Repo Evidence / Role
Electron shell	electron/app.js starts server, opens BrowserWindow, serves web-frontend/dist.
React frontend	web-frontend/src renders Devices, WA, Share, Received, Batch, Display tabs.
HTTP API	desktop-host/src/server/http-api.js exposes device, session, group, master, batch, share, warm-up APIs.
WebSockets	ws-server.js routes /ws/control, /ws/video, and /ws/signaling; forwards device events and H.264 frames.
ADB + scrcpy	device-manager.js polls ADB; session-manager and scrcpy-client manage per-device mirror/control sessions.
Persistence	Nicknames, groups, and warm-up state use JSON/userData paths; Docker maps host-data volume.

Data flow: Android devices -> ADB/scrcpy sessions -> backend events/video -> WebSocket clients -> React VideoPlayer canvas; UI input -> /ws/control -> sessionManager -> scrcpy control/UHID -> device.

How To Run

- Prereqs from README: Windows 10/11, ADB on PATH, Android devices with USB Debugging, Node.js 18+ for development.
- **Install deps:** npm run setup
- **Run dev desktop app:** npm run dev
- **Build Windows installer/portable app:** npm run build
- **Alternative Docker path:** docker-compose.yml defines desktop-host on port 3001 and web-frontend on port 80; host ADB server setup is documented inline.

All claims are drawn from repo files present at generation time; no external sources used.